

УНИВЕРСИТЕТ ИТМО

Факультет прикладной информатики

Направление подготовки 09.03.03 Прикладная информатика

Дисциплина «Бекэнд-разработка»



ОТЧЕТ по

лабораторной работе 2

“Межсервисное взаимодействие через RabbitMQ”

Выполнил:

Таипов Т.А.

Группа:

К3441

Преподаватель:

Добряков Д.И.

Санкт-Петербург

2026 г.

Содержание

ВВЕДЕНИЕ	2
1 Задача	2
2 Ход работы	2
2.1 Подключение RabbitMQ	2
2.2 Реализация producer в Application Service	2
2.3 Реализация consumer в Job Service	3
2.4 Дополнительная настройка сетевого взаимодействия	3
2.5 Схема взаимодействия	3
3 Вывод	3

ВВЕДЕНИЕ

В лабораторной работе реализовано асинхронное межсервисное взаимодействие в микросервисном приложении `jobboard-microservices` с использованием RabbitMQ.

1 ЗАДАЧА

Текст задачи на лабораторную/практическую работу:

- подключить и настроить RabbitMQ/Kafka;
- реализовать межсервисное взаимодействие посредством RabbitMQ/Kafka.

2 ХОД РАБОТЫ

2.1 Подключение RabbitMQ

В качестве брокера сообщений выбран RabbitMQ. В файл `jobboard-microservices/docker-compose.yml` добавлен сервис:

- `rabbitmq:3-management`;
- порт 5672 для AMQP;
- порт 15672 для web-интерфейса управления;
- подключение к общей сети `jobboard-network`.

Для сервисов `application-service` и `job-service` добавлены переменные окружения:

- `RABBITMQ_URL=amqp://guest:guest@rabbitmq:5672`;
- `RABBITMQ_EXCHANGE=jobboard.events`;
- для `job-service` дополнительно очередь и routing key: `RABBITMQ_APPLICATION_QUEUE` и `RABBITMQ_APPLICATION_ROUTING_KEY`.

2.2 Реализация producer в Application Service

В `application-service` добавлен модуль `src/messaging/rabbit.ts`, который:

- устанавливает соединение с RabbitMQ;
- выполняет `assertExchange` для `jobboard.events` (тип `topic`);
- публикует сообщения через функцию `publishEvent(...)`.

После успешного создания отклика (`POST /api/applications`) сервис публикует событие `application.submitted` с данными:

- `id` отклика;

- `id` вакансии;
- `id` кандидата и профиля;
- статус и время создания.

2.3 Реализация consumer в Job Service

В `job-service` добавлен модуль `src/messaging/rabbit.ts`, который:

- подключается к RabbitMQ;
- создает/проверяет exchange `jobboard.events`;
- создает очередь `job-service.application-submitted`;
- связывает очередь с ключом `application.submitted`;
- запускает consumer с подтверждением сообщений (`ack`).

Запуск consumer интегрирован в `job-service/src/server.ts` при старте сервиса. При получении события сервис пишет в лог информацию об отклике и вакансии.

2.4 Дополнительная настройка сетевого взаимодействия

Для корректного обращения `application-service` к `job-service` в Docker-сети проверка вакансии переведена на адрес по имени сервиса:

- `JOB_SERVICE_URL=http://job-service:4002`.

Это исключает зависимость от `localhost` внутри контейнера.

2.5 Схема взаимодействия

Client -> API Gateway -> Application Service

Application Service -> PostgreSQL (создание отклика)

Application Service -> RabbitMQ Exchange (`application.submitted`)

RabbitMQ Queue(`job-service.application-submitted`) -> Job Service Consumer

Job Service -> лог обработки события

3 ВЫВОД

В ходе лабораторной работы подключен и настроен RabbitMQ, а также реализовано межсервисное взаимодействие по событийной модели: `application-service` публикует событие `application.submitted`, `job-service` принимает и обрабатывает его через очередь RabbitMQ. Задача лабораторной работы выполнена.